Structure and Basic File IO

Song Liu (song.liu@bristol.ac.uk)
GA 18, Fry Building,
Microsoft Teams (search "song liu").

Today

1. Structure: Group variables together.
2. File IO functions: Operating on files.

Grouping

Grouping Variables

- We have learned how to declare **individual** variables:
  - `double pi = 3.141592654;`
  - `int studentID = 195235;`
- In many applications, variables are bundled together and should be dealt with as a group
- For example:
  - `studentID` are commonly associated with other basic information, such as `name`, `age`, `grade` etc.
  - An administrative software would like to handle these variables as a group of variables, rather than individual variables.

Example: Vector Operations

- Print vector

```
void print(double vec1[], int len1){
    for(int i = 0; i<len1; i++){
        printf("%.2f ", vec1[i]);
    }
    printf("\n");
}
```

- Vector dot product

```
 double dot(double vec1[], int len1,
            double vec2[], int len2){
     double sum = 0;
     for(int i = 0; i<len1; i++){
         sum += vec1[i]*vec2[i];
     }
     return sum;
 }
```

Example: Vector Operations

- Declare and initialize vectors (arrays)

```
int len1 = 3; int len2 = 3;
double *vec1 = malloc(len1*sizeof(double)); //heap mem
double *vec2 = malloc(len2*sizeof(double)); //heap mem
```

- Call vector functions

```
print(vec1, len1);
print(vec2, len2);
double d = dot(vec1, len1, vec2, len2);
// ... free heap memory, don't forget!
```

- The array and its length are always **linked** in our code.
- Having to write the array and its length in every function seems unnecessarily complicated!

Example: Vector Operations

- Would be nice to call functions like this:

```
//v1 and v2 contains elems and its length
print(v1);
double d = dot(v1, v2);
```

- Python programs work in this way:

```
>>> v1 = [1, 2, 3]
>>> print(v1)
[1, 2, 3]
```

- How can we do it in C?

Grouping Variables

- Since `len1, vec1` are all variables describing the vector, we can group them together.
- Introducing a C language feature: **Structure**.
- A structure groups several related variables into **a single entity.**

Structure

- A structure is a variable that contains sub-level variables.
- Syntax for defining a structure:

```
struct structure_name{
    data_type variable1;
    data_type variable2;
    ...
}; //Do not forget the ```;``` at the end!
```

- Syntax for declaring a structure variable

```
struct structure_name struct_variablename;
```

- Syntax for referencing a sub-level variable contained in a structure variable

```
struct_variablename.variable1
```

Example: Student

- First, let us study a toy example.
- Define a structure, `student` which contains three sub-level variables `ID`, `name` and `grade`.

Example: Student

Define a structure "student"

```
struct student{
    int ID;
    char *name;
    int grade;
};
```

Declare a structure variable and initialize it:

```
struct student song;
song.ID = 1024;
song.name = "song liu";
song.grade = 70;
// print out song's name
printf("%s\n", song.name);
```

Example: Student

When initializing a structure variable, you can use a syntax that is similar to the array initialization:

```
struct student song = {1024, "song liu", 70};
printf("%s\n", song.name);
//displays: song liu
```

It only works when initializing! You **CANNOT** do

```
struct student song;
song = {1024, "song liu", 70};// COMPILATION ERROR!!!
```

Example: Student (full code)

```c
#include <stdio.h>
struct student{
    int ID;
    char *name;
    int grade;
}; //Define a structure before you use it!!

void main(){
    struct student song; //declare a student variable
    //initialize
    song.ID = 1024;
    song.name = "song liu";
    song.grade = 70;

    printf("%s\n", song.name); //displays "song liu"

    //declare + initialize in one line.
    struct student jack = {1111, "jack", 80};

    printf("%s\n", jack.name); //displays "jack"
}
```

Passing by Value

- Structures are **passed by value.**
    - This is different from arrays, passed by reference!

```c
#include<stdio.h>
struct student{
  int ID;
  char *name;
  int grade;
};

void hack(struct student s){
  s.grade = 9999; //trying to hack the score!
}
void main(){
  struct student song = {1234, "song liu", 70};
  hack(song);
  printf("%d\n", song.grade);//display 70, not 9999!
}
```

Example: Vector

- Define a `vector` structure

```
struct vector{
  int len;
  int *elements; //pointing to the array
                 //stores elements of the vector.
};
```

- Declare a `vector` structure variable and initialize it

```
struct vector v;
v.len = 10;
// allocate heap memory for the vector
v.elements = malloc(v.len * sizeof(int));
```

- Or in one line

```
struct vector v = {10, malloc(10 * sizeof(int))};
```

Example: Vector Operations 2.0

- Write a function that prints a vector, using `struct vector` as the input.

```
void print(struct vector v){
    // v.len contains the length of the vector!
    for (int i=0; i<v.len; i++){
        printf("%d ", v.elements[i]);
    }
    printf("\n");
}
```

- Finally, we can call the `print` like this:

```
print(v);
```

- The interface and usage of function `print` is much cleaner the earlier version.

Input and Output (IO)

Overview

- In C, all IO operations are handled by function calls.
  - We have already encountered one such function
  - `printf(...)`
- Thanks to the abstraction of hardware, whatever IO devices you are using, these function calls are exactly the same.
- Today, the IO functions in C still inspire IO function designs in other programming languages.
- Here, we are going to focus on File IO.

Open a File `fopen`

- Usage: `FILE *fopen(char *filename, char *mode)`
  - `filename` : string, file name.
  - `mode` : access mode, can be
    - `"r"` : read-only, file must exist.
    - `"w"` : write, create an empty one if file does not exist.
    - `"r+"` : read and write, file must exist.
    - `"w+"` : read and write, create an empty file if file does not exist.
    - `"a"` : appending, create an empty one if file does not exist.
    - `"a+"` : appending and reading, create an empty

Open a File `fopen`

- Usage: `FILE *fopen(char *filename, char *mode)`
- `fopen` returns a pointer to a `FILE` structure.
  - You do not need to understand what `FILE` structure is. The definition of `FILE` structure is not visible to you.
  - This pointer is needed for further operations on the file.

Close a File `fclose`

- After read/write operations on a file, you MUST close it.
- Usage: `int fclose(FILE * file)`
    - The input is the pointer you obtained from `fopen`.

Stream

- The design of C's IO functions are heavily influenced by the IO devices in the 60s, 70s.
    - These devices are mostly sequential and can move along one direction, such as tapes.
    - You can only read/write one byte after another.
    - Like a riding boat in a river...
- The abstraction of such devices is called IO Stream.
    - IO functions can only read or write "the next thing" in the stream.
    - The `FILE *` pointer indicates our current position in the stream.

Read the next Byte `fgetc`

- `int fgetc(FILE *file)`
  - `file` : the pointer you obtained using `fopen` .
  - Returns the next byte in the stream, as an `int` variable.

Write the next Byte `fputc`

- `int fputc(int byte, FILE *file)`
  - `file` : the pointer you obtained using `fopen` .
  - `byte` : the byte to be written.
- When using `fgetc` or `fputc` , you need to set the `mode` in `fopen` to be `wb` , `rb` or `ab` , where `b` stands for binary.

Read the next Line `fgets`

- `char *fgets(char *line, int max, FILE* file)`
  - `line` : a pointer to an char array where the line is going to be stored.
  - `max` : the maximum number of character to be read.
  - `file` : the pointer you obtained using `fopen` .

Write formatted string `fprintf`

- `int fprintf(FILE *file, char *line, variables)`
  - `file` : the pointer you obtained using `fopen` .
  - `line` : the formatted string containing specifiers, like the one in `printf` .
  - `variable` : variables corresponds to the specifiers in `line` , like in `printf` .
- `fprintf(file, "pi is %.2f.\n", 3.14)`
  - Write a line "pi is 3.14." to `file`

Example: Reading Lines from File

```c
#include <stdio.h>
void main()
{
    FILE *f = fopen("poem.txt", "r");
    char line[1024];
    while (1){ // loop forever until reach the end
        fgets(line, 1024, f); // read the next line
        if (feof(f)){
            break;// stop the loop if we are at
        // the end of the file
        }

        //print the line to screen
        printf("%s", line);
    }
    fclose(f);
}
```

Is this the end of file? `feof`

```
while (1){
  // ...
    if (feof(f)){
        break;
    }
  //...
}
```

- As we read/write the next byte/line, we push the `FILE` pointer further down the IO stream until it reaches the End of File (EOF).
    - We can test whether EOF has been reached using the `FILE` pointer.
- `int feof(FILE *file)`
    - `file` : the pointer you obtained using `fopen` .

Conclusion

- Structure is a mechanism in C that groups related several variables together as a single entity.
    - Student example
    - Vector example
- Input and Output (IO)
    - File IO in C is handled as IO streams.
    - Read/Write a byte `fgetc`, `fputc`.
    - Read/Write a line `fgets`, `fprintf`.

Homework 8.1

1. Download the lab file.
2. Read `student.c` and see how a structure is defined and sub-level variables are initialized and accessed.
3. Define a `print` function takes a `struct student` variable and print out student's information in a easy to read manner. For example:

- `ID: ..., Name: ..., Grade: ...`

Homework 8.2

1. Open `vector_ops2.c` and see how the `vector` structure is defined.
2. Write the `dot` function that takes two vectors u, v and compute their dot product $u \cdot v$.
3. Write the `norm` function that computes the norm of a vector v.
   $\text{norm}(v) = \sqrt{v \cdot v}$
4. Write the `dist` function that computes the euclidean distance between its input u and v:
   $\text{dist}(u, v) = \sqrt{u \cdot u + v \cdot v - 2u \cdot v}.$
5. Test your functions in `main` with provided test cases.

Homework 8.3 (submit)

Now, let us adapt vector structure to create a matrix structure.

1. Create a new c file called `matrix_ops.c`.
2. Create a structure called `matrix`, containing three sub-level variables: `m`, `n`, `elements`, where
    i. `elements` is an integer pointer, pointing to an array which stores a matrix $A \in \mathbb{N}^{m \times n}$ in row-major order.
    ii. `m` and `n` are integer variables.
3. Declare a `matrix` structure variable `m1`, and initialize it with a 2 by 3 matrix:

$$A = \begin{bmatrix} 1, 2, 3 \\ 4, 5, 6 \end{bmatrix}.$$

4. Write a function `get_elem`, which takes three inputs `m,i,j`. i, j are **zero-based indices** and `m` is a matrix structure. It returns the i, j-th entry of A stored in `m`.
5. Write a function `void print(struct matrix m),` which takes a single matrix structure variable as input and prints out all matrix elements.

- Use `get_elem` function that you have just defined.

5. Test your `print` using `m1` matrix you have just created.